昵称： KillerAery
园龄： 8年5个月
粉丝： 460
关注： 22

+加关注

游戏开发中的噪声算法

目录

- 噪声
  - 随机性
  - 哈希性
  - 平滑性(连续性)
- Value噪声
  - 原理
  - 实现(二维)
- 柏林噪声
  - 原理
  - 实现(二维)
- Simplex噪声
  - 原理
  - 实现（二维）
- 可平铺的噪声
  - 翻转纹理
  - 对高维度的圆采样
- 分形噪声
- 结语
- 参考

噪声

噪声是游戏编程的常见技术，广泛应用于地形生成，图形学等多方面。

◇随笔分类 (35)

C# 笔记(3)

C++ 笔记(5)

Gameplay 实验室(...

UE4 C++ 入门(5)

程序语言理论 TPL(2)

计算机考研(1)

游戏动画(1)

游戏架构&游戏设...

游戏开发工具(3)

游戏算法&数据结...

◇阅读排行榜

1. 空间数据结构(四叉树/八叉树/BVH树/BSP树/k-d树)(65460)

2. C++ Sqlite3的基本使用(42063)

3. JPS/JPS+ 寻路算法(33973)

4. 透彻理解C++11移动语义：右值、右值引用、std::move、std::forward(32885)

5. 游戏AI之决策结构—行为树(19491)

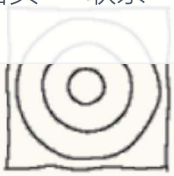
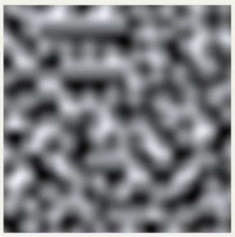
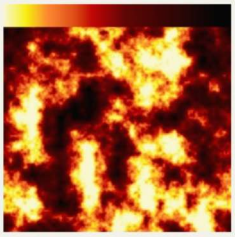
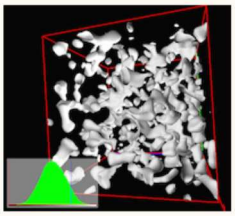
6. 游戏架构设计：面向数据编程（DOP）(17862)

7. 游戏AI之路径规划(16529)

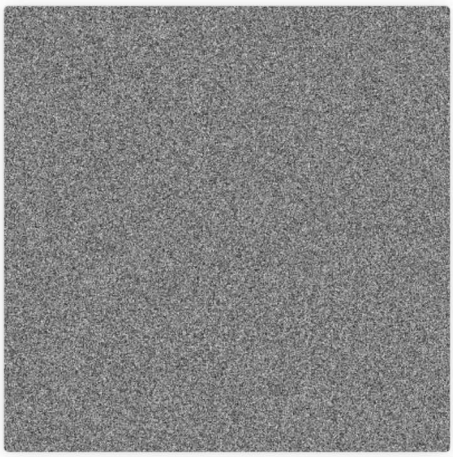
8. 中科大软件学院2021计算机考研有感（已上岸）(16007)

9. 基于 Probe 的实时全局光照（Probe-based Global Illumination）(15349)

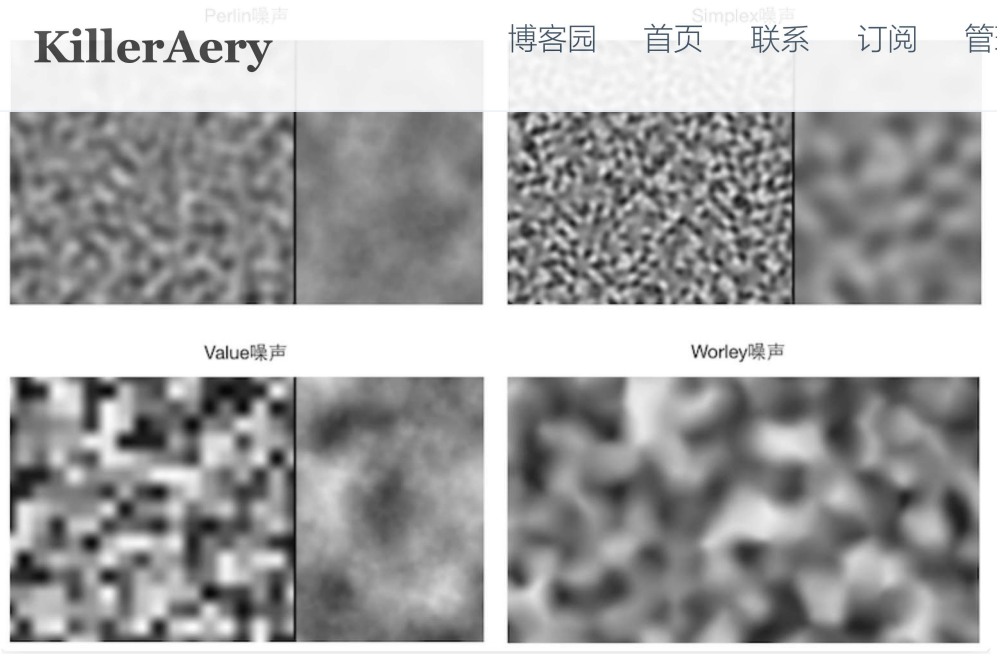
10. C++11 智能指针的深度理解(15186)

噪声维度	原始噪声（灰阶）	游戏应用
1		 使用噪声进行偏移，制造手绘效果
2		 制造火焰燃烧特效
3		 最普遍的运用莫过于生成游戏里的随机地形

那么为什么要引入噪声这个概念呢？在程序中，我们经常使用直接使用最简单的rand()生成随机值，但它的问题在于生成的随机值太“随机”了，得到的值往往总是参差不齐，如下图使用随机值作为像素点的黑白程度：



而使用噪声，我们得到的值看起来虽然随机但平缓，这种图也看起来更自然和舒服：



而根据wiki，现在噪声类型已经有很多种类：

类别	名称
基于晶格的方法（Lattice based）	Perlin噪声，Simplex噪声，Wavelet噪声，Value噪声
基于点的方法（Point based）	Worley噪声

本文主要说明Value噪声，Perlin噪声，Simplex噪声这三种常见的噪声。

随机性

随机性是噪声的基础，不必多说。

哈希性

在《Minecraft》里，由于世界是无限大的，它以"Chunk"区块（16×16×256格子）为单位，只加载玩家附近的区块。也就是说，当玩家在移动时，它会卸载远离的区块，然后加载靠近的区块。

一个问题是，当玩家离开一个区块时，进入第二个区块，然后又回到第一个区块，此时玩家期望看到的第一个区块和之前看到的保持一致。例如，输入1时得到0.3，输入2时得到0.7，当再次输入1时预期得到0.3。

因此噪声的一个重要性质是哈希性（可哈希的）。

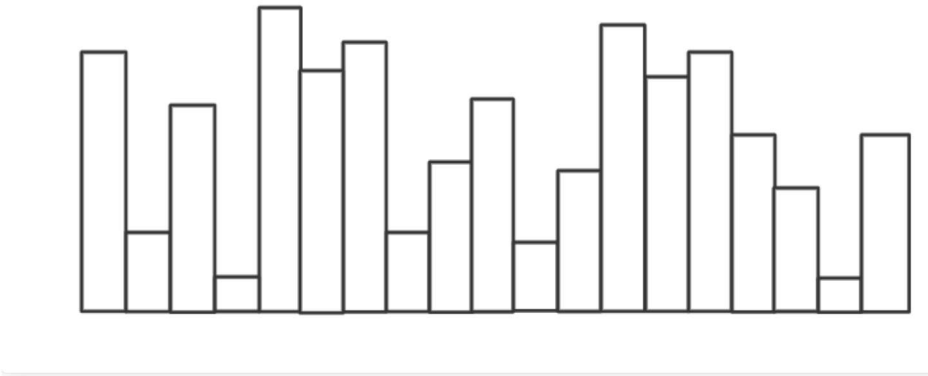
> 尽管使用输入值作为srand()的参数来设置rand()的种子，从而达到哈希效果也是可行的。然而最好花点时间写一个自己的哈希函数，使其简易使用而且也不破坏程序其他地方使用rand()的效果。

```
//一个随机性的哈希函数
unsigned int hash11(int position){
const unsigned int BIT_NOISE1 = 0x85297A4D;
const unsigned int BIT_NOISE2 = 0x68E31DA4;
const unsigned int BIT_NOISE3 = 0x1B56C4E9;
unsigned int mangled = position;
mangled *= BIT_NOISE1;
mangled ^= (mangled >> 8);
mangled += BIT_NOISE2;
mangled ^= (mangled << 8);
```

> hash11的11代表输入一维坐标，输出一维值。类似的hash22代表输入二维坐标，输出二维值。
> 若要了解更多随机性哈希函数实现，可参考下面两个shadertoy的代码：
>
> - https://www.shadertoy.com/view/4sc3z2
> - https://www.shadertoy.com/view/4djSRW

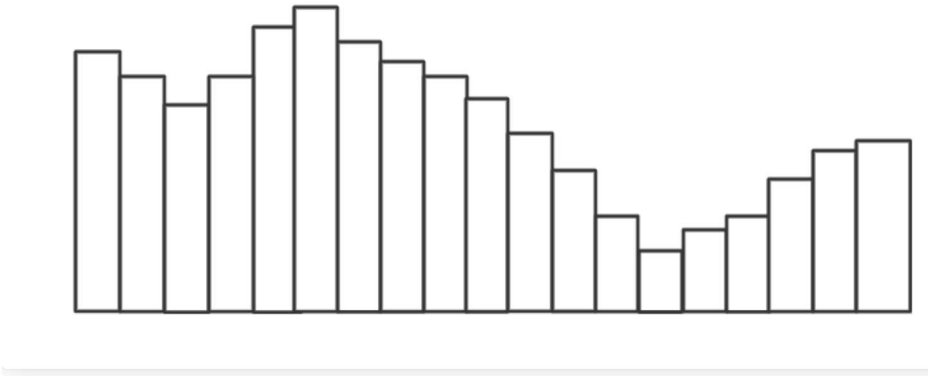
平滑性(连续性)

对一个随机生成地形来说，如果简单的使用随机和哈希组合，
那么容易得到下图（以一维地图举例，x轴为位置，y轴为地形高度）：



容易看出的问题是，由于随机的杂乱无章，地形非常的参差不齐，这可不是一个自然的地形。

我们期望得到的地形不仅随机还应该是平滑的，这样才显得自然，如下图：



为了达到连续性，自然想到利用插值函数进行插值，常见的插值方法有：线性插值、缓和曲线插值

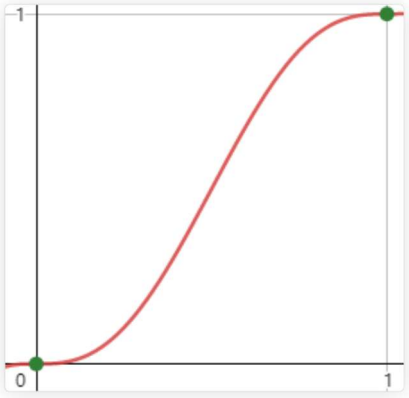
Value噪声

Value噪声是最简单的一种噪声，其主要思路是定义若干个顶点且每个顶点含有一个随机值（以顶点坐标作为参数通过哈希运算得到的），该随机值会周围坐标产生影响，越靠近顶点则越容易受该顶点影响（输出值越接近顶点随机值）。当需要求某个坐标的输出值时，需要将该坐标附近的各个顶点所造成的影响值进行叠加，从而得到一个总值并输出之。

原理

2.输入一个点（二维空间的话就是2D坐标），我们找到它所在晶格的顶点（二维下有4个，三维下有8个，N维下有2^n个），并经过哈希运算得到这些顶点的伪随机值。

3.根据这些顶点的伪随机值，使用插值函数计算出输入点的输出值。对于插值函数的权重，我们还需要使用缓和曲线（ease curves）来计算这些伪随机值的权重和。在原始的Perlin噪声实现所使用的缓和曲线是$s(t) = 3t^2 - 2t^3$，在2002年的论文中，Perlin又改进为$s(t) = 6t^5 - 15t^4 + 10t^3$。

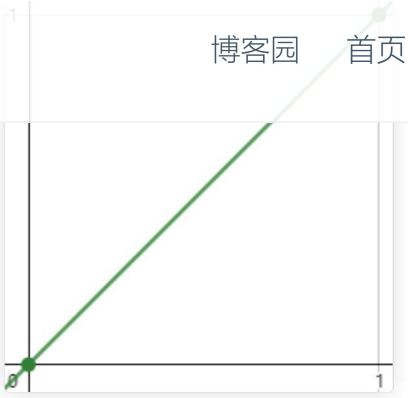


图：$s(t) = 6t^5 - 15t^4 + 10t^3$ 缓和曲线

```
//缓和曲线计算权重
float fade(float t){
  //return t * t * (3.0 - 2.0 * t); 使用3t^2-2t^3曲线也是可行的
  return t * t * t * (t * (t * 6 - 15) + 10); // 6t^5-15t^4+10t^3
}


//二维空间插值函数
//x、y分别为输入点在所在晶格的相对位置（均为0.0~1.0）
//vertexRandom[4]为四个顶点的为随机值
float interpolation(float x,float y,float vertexRandom[4]){
  //缓和曲线插值
  return lerp(
    lerp(vertexRandom[0],vertexRandom[1],fade(x)),
    lerp(vertexRandom[2],vertexRandom[3],fade(x)),
    fade(y));
}
```

> 若直接使用线性插值，其一阶导在晶格顶点处（即t = 0或t = 1）不为0，会造成明显的不连续性。$s(t) = 3t^2 - 2t^3$ 在一阶导满足连续性，$s(t) = 6t^5 - 15t^4 + 10t^3$ 在二阶导上仍然满足连续性。
> 所以实际上两种缓和曲线都是可用的，如果需要压榨开销，则使用 $s(t) = 3t^2 - 2t^3$ 。
> 对于预计算，例如程序化生成凹凸纹理（置换纹理），使用$s(t) = 6t^5 - 15t^4 + 10t^3$ 的效果更好。



图：$s(t) = t$ 线性直线

```
float interpolation(float x,float y,float vertexRandom[4]){
  //双线性插值
  return lerp(
    lerp(vertexRandom[0],vertexRandom[1],x),
    lerp(vertexRandom[2],vertexRandom[3],x),
    y);
}
```

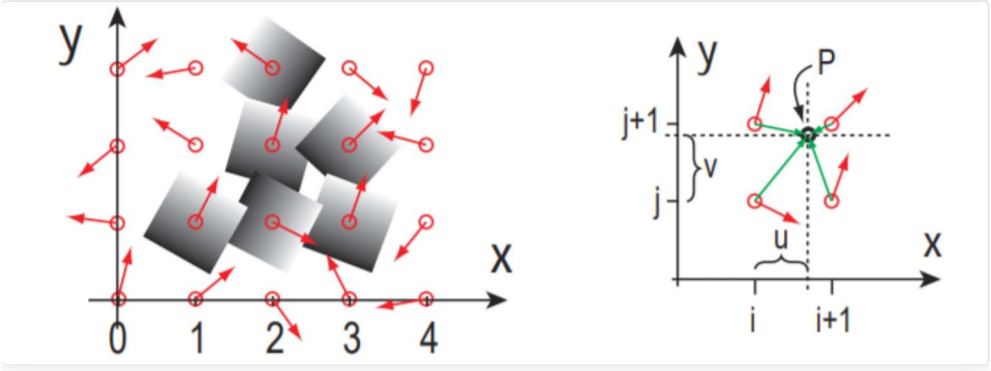
实现(二维)

```
int valueNoise(Vector2 p){
  //晶格以1为长度单位，通过向下取整可以确定p点所在晶格
  //注意：不应使用转变整型，因为负数的转整型是向上取整，而正数则是向下取整，这可能会导致(-1~(
  Vector2 pi = Vector2(floor(p.x),floor(p.y));
  //找到对应晶格的四个顶点坐标
  Vector2 vertex[4] = {{pi.x,pi.y},{pi.x+1,pi.y},{pi.x,pi.y+1},{pi.x+1,pi.y+1}};
  //通过hash21函数得出坐标对应的随机值
  float vertexRandom[4] = {{hash21(vertex[0])},{hash21(vertex[1])},{hash21(vertex[2])
  //wx、wy代表p点的权重，实际就是以(0.0~1.0)的范围表示在晶格中的位置比例
  float wx = (p.x-pi.x))/1.0f;
  float wy = (p.y-pi.y))/1.0f;
  //插值
  return interpolation(wx,wy,vertexRandom);
}
```

柏林噪声

谈起噪声，最著名的且最常用的莫过于Perlin噪声，Perlin噪声的名字来源于它的创始人Ken Perlin。

在理解了上面Value噪声后，我们再来看看柏林噪声的主要想法：
定义若干个顶点且每个顶点含有一个随机梯度向量，这些顶点会根据自己的梯度向量对周围坐标产生势能影响，沿着顶点的梯度方向越上升则势能越高。当需要求某个坐标的输出值时，需要将该坐标附近的各个顶点所造成的势能进行叠加，从而得到一个总势能并输出之。



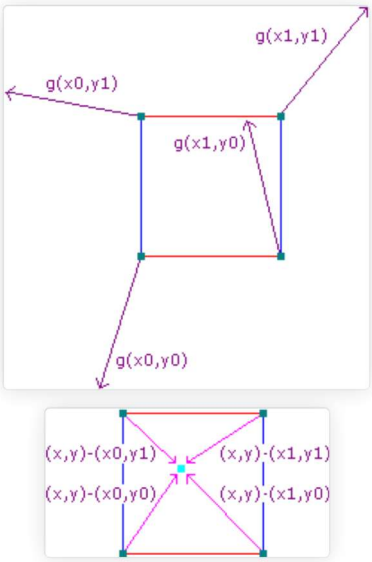
我们给顶点赋予一个随机性的哈希函数，输入一个坐标可以得到一个随机向量，满足上述随机性和
此外，由于势能是沿着梯度方向渐变的，所以很容易得到平滑性。

原理

和Value噪声一样，它也是一种基于晶格的噪声，也需要三个步骤：

1.首先定义一个晶格结构，每个晶格的顶点有一个随机的梯度向量。对于二维的Perlin噪声来说，晶格结构就是一个平面网格（通常是正方形），三维的就是一个立体网格（通常是正方体）

2.输入一个点（二维空间的话就是2D坐标），我们找到它所在晶格的顶点（二维下有4个，三维下有8个，N维下有2^n个），并经过哈希运算得到这些顶点的梯度向量（随机向量）；接着计算该点到各个晶格顶点的距离向量，再分别与顶点代表的梯度向量做点乘，得到2^n个梯度值结果

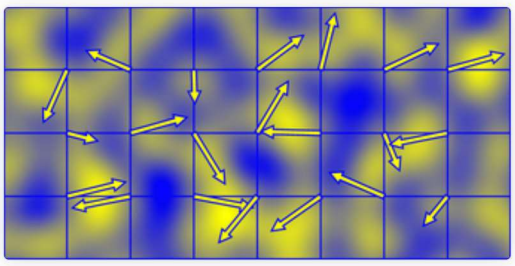



```
//点乘
float dot(Vector2 v1,Vector2 v2){
  return v1.x*v2.x+v1.y*v2.y;
}

//求梯度值（本质是求顶点代表的梯度向量与距离向量的点积）
float grad(Vector2 vertex, Vector2 p)
{
  return dot(hash22(vertex), p);
}
```

3.使用缓和曲线来计算它们的权重和（同样的，可以是$s(t) = 3t^2 - 2t^3$，也可以是s(t)=$6t^5 - 15t^4 + 10t^3$）

下图通过颜色差异显示了由2D柏林噪声生成的各像素点的值：



实现(二维)

```
//二维柏林噪声
float perlin(Vector2 p)
{
  //向量两个纬度值向下取整
  Vector2 pi = Vector2(floor(p.x),floor(p.y));
  //找到对应晶格的四个顶点坐标
  Vector2 vertex[4] = {{pi.x,pi.y},{pi.x+1,pi.y},{pi.x,pi.y+1},{pi.x+1,pi.y+1}};
  //通过grad函数得出坐标对应的随机值
  float vertexRandom[4] = {grad(vertex[0],p),grad(vertex[1],p),grad(vertex[2],p),grac
  //wx、wy代表p点的权重，实际就是以(0.0~1.0)的范围表示在晶格中的位置比例
  float wx = (p.x-pi.x))/1.0f;
  float wy = (p.y-pi.y))/1.0f;
  //插值
  return interpolation(wx,wy,vertexRandom);
}
```

---

gard函数另一个更快的实现方式，它与标准实现方式的区别是：晶体顶点是从若干个梯度向量里随机选择一个向量而不是产生一个随机向量，这样做可以预先计算好求梯度值时各项的系数。因此我们只需这样重写一下grad函数：

```
//求梯度值（本质是求顶点代表的梯度向量与距离向量的点积）
float grad(Vector2 vertex, Vector2 p)
{
    switch(hash21(vertex) % 4)
    {
      case 1: return  p.x + p.y;  //代表梯度向量(1,1)
      case 2: return -p.x + p.y;  //代表梯度向量(-1,1)
      case 3: return  p.x - p.y;  //代表梯度向量(1,-1)
      case 4: return -p.x - p.y;  //代表梯度向量(-1,-1)
      default: return 0; // never happens
    }
}
```

> 这里示例提供了4个可选的随机向量，实际上这个数量是偏少的，如果想要更加多样的效果，建议在实现时多提供些可选的随机向量。

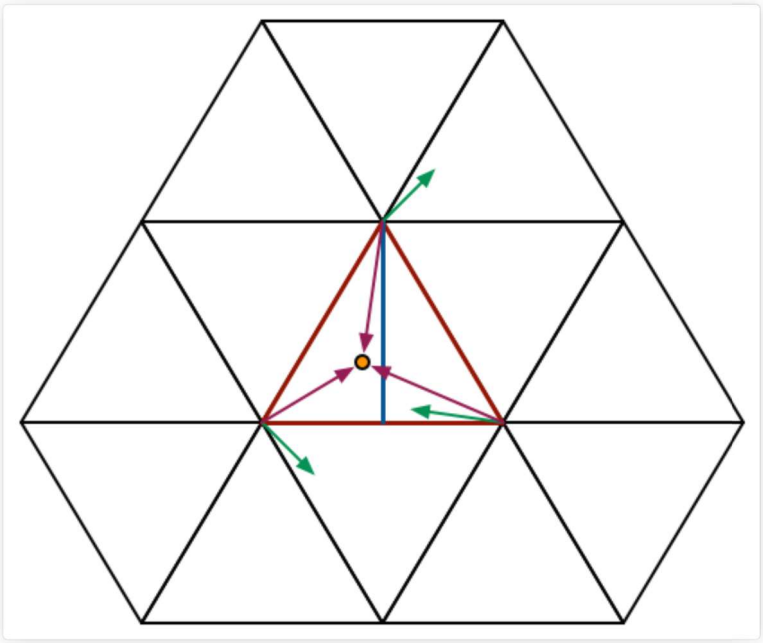
Simplex噪声

Simplex噪声也是一种基于晶格的梯度噪声，它和Perlin噪声在实现上唯一不同的地方在于，它的晶格并不是方形（在2D下是正方形，在3D下是立方体，在更高纬度上我们称它们为超立方体，hypercube），而是单形（simplex）。

> 通俗解释单形的话，可以认为是在N维空间里，选出一个最简单最紧凑的多边形，让它可以平铺整个N维空间。我们可以很容易地想到一维空间下的单形是等长的线段，把这些线段收尾相连即可铺满整个一维空间。在二维空间下，单形是三角形，我们可以把等腰三角形连接起来铺满整个平面。三维空间下的单形就是四面体。更高维空间的单形也是存在的。



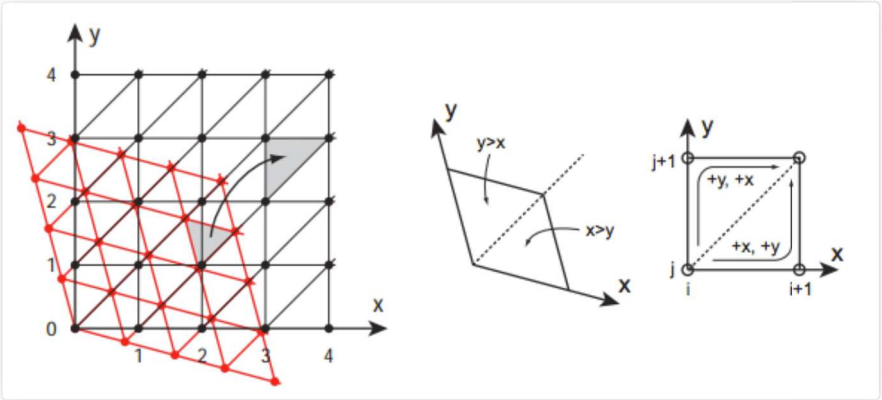
总结起来，在n维空间下，超立方体的顶点数目是2^n，而单形的顶点数目是$n+1$，这使得我们在计算梯度噪声时可以大大减少需要计算的顶点权重数目。



一个潜在的问题是如何找到输入点所在的单形。

在计算Perlin噪声时，判断输入点所在的正方形是非常容易的，我们只需要对输入点下取整即可找到。

对于单形来说，我们需要对单形进行坐标偏斜（skewing），把平铺空间的单形变成一个新的网格结构，这个网格结构是由超立方体组成的，而每个超立方体又由一定数量的单形构成：



我们之前讲到的单形网格如上图中的红色网格所示，它们有一些等边三角形组成（注意到这些等边三角形是沿空间对角线排列的）。经过坐标倾斜后，它们变成了后面的黑色网格，这些网格由正方形组成，每个正方形是由之前两个等边三角形变形而来的三角形组成。这个把N维空间下的单形网格变形成新网格的公式如下：

$$x' = x + (x + y + \dots) \cdot K1$$
$$y' = y + (x + y + \dots) \cdot K1$$

体，在二维下即为正方形。

原理

1.坐标偏斜：把输入点坐标进行坐标偏斜。

$x' = x + (x + y + \dots) \cdot K1$

$y' = y + (x + y + \dots) \cdot K1$

其中，$K1 = \frac{\sqrt{n+1}-1}{n}$

2.找到顶点：对偏斜后坐标下取整得到输入点所在的超立方体$xi = floor(x')$，$yi = floor(y')$,...我们还可以得到小数部分$xf = x' - xi, yf = y' - yi$,...

我们把之前得到的(xf,yf,...)中的数值按降序排序，来决定输入点位于变形后的哪个单形内。这个单形的顶点是由按序排列的(0, 0, ..., 0)到(1, 1, ..., 1)中的n+1个顶点组成，共有n!种可能性。

我们可以按下面的过程来得到这n+1个顶点：从零坐标(0, 0, ..., 0)开始，找到当前最大的分量，在该分量位置加1，直至添加了所有分量。这一步的算法复杂度即为排序复杂度$O(n^2)$。

> 例如，对于二维空间来说，如果xf,yf满足xf>yf，那么对应的3个单形坐标为：首先找到(0, 0)，由于x分量比较大，因此下一个坐标是(1, 0)，接下来是y分量，坐标为(1, 1)；对于三维空间来说，如果xf,yf,zf满足xf>zf>yf，那么对应的4个单形坐标位：首先从(0, 0, 0)开始，接下来在x分量上加1得(1, 0, 0)，再在z分量上加1得(1, 0, 1)，最后在y分量上加1得(1, 1, 1)。

3.梯度选取：我们在偏斜后的超立方体网格上获取该单形的各个顶点的伪随机梯度向量。

4.变换回单形网格里的顶点：我们首先需要把单形顶点变回到之前由单形组成的单形网格。这一步需要使用第一步公式的逆函数来求得：

$x = x' + (x' + y' + \dots) \cdot K2$

$y = y' + (x' + y' + \dots) \cdot K2$

其中，$K2 = \frac{\frac{1}{\sqrt{n+1}}-1}{n}$

5.贡献度取和：我们由此可以得到输入点到这些单形顶点的位移向量。这些向量有两个用途，一个是为了和顶点梯度向量点乘，另一个是为了得到之前提到的距离值dist，来据此求得每个顶点对结果的贡献度：

$(r^2 - |dist|^2)^4 \times dot(dist, grad)$

实现（二维）

```
float simplexNoise(Vector2 p)
{
  const float K1 = 0.366025404; // (sqrt(3)-1)/2;
  const float K2 = 0.211324865; // (3-sqrt(3))/6;
  //坐标偏斜
  float s = (p.X + p.Y) * K1;
  Vector2 pi = Vector2(floor(p.X+s),floor(p.Y+s));
  float t = (pi.X + pi.Y) *K2;
  Vector2 pf = p-(pi-t*Vector2::UnitVector);
  Vector2 vertex2Offset = (pf.X < pf.Y) ? Vector2(0, 1) : Vector2(1, 0);

  //顶点变换回单行网格空间
  Vector2 dist1 = pf;
  Vector2 dist2 = pf - vertex2Offset + K2 * Vector2::UnitVector;
  Vector2 dist3 = pf - Vector2(1,1) + 2 * K2 * Vector2::UnitVector;

  //计算贡献度取和
  float hx = 0.5f - Vector2::DotProduct(dist1, dist1);
```

```
    float hy = 0.5f - Vector2::DotProduct(dist2, dist2);
    float hz = 0.5f - Vector2::DotProduct(dist3, dist3);

    hx=hx*hx*hx*hx;
    hy=hy*hy*hy*hy;
    hz=hz*hz*hz*hz;

    //结果范围是[-1,1]
    return 70*(
          hx*Vector2::DotProduct(dist1, hash22(pi))
          +hy*Vector2::DotProduct(dist2, hash22(pi + vertex2Offset))
          +hz*Vector2::DotProduct(dist3, hash22(pi + Vector2(1,1)))
         );
}
```

> r^2取0.5的原因是因为要求经过第一步坐标偏斜后得到的网格宽度为1，因此我们可以倒推出在变形前单形网格中每个单形边的边长为$\sqrt{\frac{2}{3}}$，这样一来单形每个顶点到对面边的距离（即高）的长度为$\frac{\sqrt{2}}{2}$，它的平方即为0.5。很奇妙的是，不仅是二维，在其他维度下，每个单形顶点到对面边/面的距离都是0.5。

虽然理解上Simplex噪声相比于Perlin噪声更难理解，但由于它的效果更好、速度更优，因此很多情况下会替代Perlin噪声。

而且高维的噪声并不少见，例如对于常见的二维噪声纹理，我们可以额外引入时间分量，变成一个2D纹理动画（三维噪声），用于火焰纹理动画等..
对于常见的三维噪声纹理，引入额外的时间分量，就可以变成一个3D纹理动画（四维噪声），用于3D云雾动画等..
当我们需要一个可循环无缝衔接的动画时（见下文可平铺的噪声），那噪声又要提高一个维度。

可平铺的噪声

可平铺的噪声就是指那些可以tiling的、seamless的噪声，因为很多时候我们想要让噪声纹理可以无缝连接，例如在生成地形时。按照我们之前提到的方法直接产生噪声，得到的噪声纹理其实是不可以平铺的，你可以看生成纹理的左右、上下其实是不一样的。那么，怎么生成可平铺的噪声纹理呢?

翻转纹理

一种低开销的trick是，首先对一张噪声纹理分别进行X轴翻转，Y轴翻转，XY轴同时翻转，从而得到新的三张噪声纹理，将它们拼接成一张大纹理，此时该大纹理为可tiling无缝的。

一个基础噪声纹理:



一个基础噪声纹理和另外三个生成的纹理拼接城的大纹理:



缺点是这样的纹理看起来可能会过于对称，影响美观。

对高维度的圆采样

一种方法是在2n维上计算n维可平铺噪声。我们以二维噪声为例，如果我们想要得到二维的无缝Perlin噪声，就需要用四维噪声算法来产生。

这种方法是思想是，由于我们想要每个维度都是无缝的，也就是当该维度的值从0变成1的过程中，0和1之间比较是平滑过渡的，这让我们想起了"圆"，绕圆一周就是对该维度的采样过程，这样就可以保证无缝了。

因此，对于二维噪声中的x轴，我们会在四维空间下的xz平面上的一个圆上进行采样，而二维噪声的y轴，则会在四维空间下的yw平面上的一个圆上进行采样。这个转化过程很简单，我们只需要使用三角函数sin和cos即可把二维采样坐标转化到单位圆上。同样，三维空间的也是类似的，我们会在六维空间下计算。这种方法不仅适用于Perlin噪声，像Worley噪声这种也同样是适合的。

Unity Wiki里二维可平铺的Simplex噪声的实现：

```
float seamlessNoise(float x, float y, float dx, float dy, float xyOffset) {
    const float PI = 3.1415926f;
    float s = x;
    float t = y;

    float nx = xyOffset + cos(s * 2.0f * Mathf.PI) * dx / (2.0f * PI);
    float ny = xyOffset + cos(t * 2.0f * Mathf.PI) * dy / (2.0f * PI);
    float nz = xyOffset + sin(s * 2.0f * Mathf.PI) * dx / (2.0f * PI);
    float nw = xyOffset + sin(t * 2.0f * Mathf.PI) * dy / (2.0f * PI);

    return noise(nx, ny, nz, nw);
}
```

> 其中，xyOffset是指在四维空间某个平面上的偏移，即这个单位圆是以xyOffset为圆心的。

该方法缺点是计算量大大增加，一般噪声的复杂度为$O(2^n)$（Simplex噪声例外，是$O(n^2)$），是指数增加的，因此比较适合预计算，例如程序化生成噪声纹理。

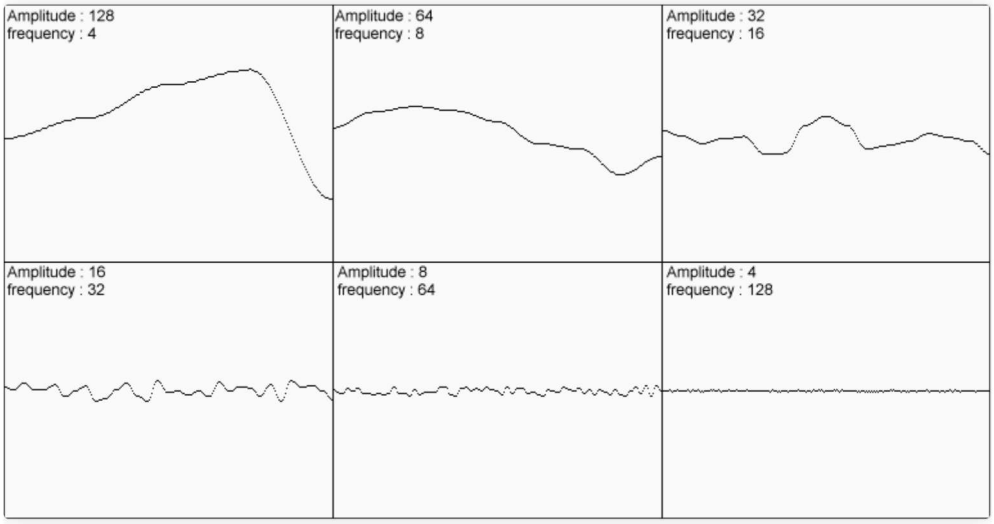
> GameDev上关于可平铺噪声的讨论，许多人分享了他们各自创造性的做法（非常有趣）：https://gamedev.stackexchange.com/questions/23625/how-do-you-generate-tileable-perlin-noise

分形噪声

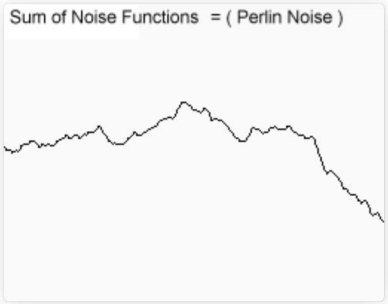
当我们使用基于晶格的噪声时，主要有两个参数可以调整：

- 频率（frequencies）：晶体格的边长（即采样间隔，例如频率越高，单位面积（特指二维）内的晶格数目越多，看起来噪声纹理"越密集"。）
- 振幅（amplitudes）：返回值的幅度范围

而在地形生成中，地形可能会有大段连绵、高耸山地，也会有丘陵和蚀坑，更小点的有岩石块，为了模拟出这样的自然噪声特性，我们可以使用不同的参数进行多几次柏林噪声计算，然后将结果叠加在一起。



将不同频率和振幅参数下的柏林噪声结果叠加在一起，我们就能得到以下结果：



很明显，这样的噪声结果更加令人信服。上面的6组噪声被称之为噪声的不同倍频(Octave)。随着倍频增大，噪声对于最终叠加噪声的影响程度变小。

那我们应该分别挑选多大的频率和振幅来进行噪声计算呢？这个可以通过persistence参数确定。Hugo Elias对persistence的定义使用如下：

$$frequency = 2^i$$
$$amplitude = persistence^i$$

简单来说，对于一维噪声，合适的组合是$Noise(x) + \frac{1}{2}Noise(2x) + \frac{1}{4}Noise(4x)+\dots$
二维噪声则是$Noise(x,y) + \frac{1}{2}Noise(2x,2y) + \frac{1}{4}Noise(4x,4y)+\dots$

公式：$\sum_{i=0}^{n} \frac{Noise(2^i point)}{2^i}$

> 以上公式i的值取决于想要倍频组数。此外至于为什么最好按1倍，2倍，4倍，8倍...的倍频叠加，是因为这样的频率叠加更贴近模拟自然界的自相似过程（可wiki查下自相似）

```
double octavePerlin(double x, double y, double z, int octaves, double persistence) {
    double total = 0;
    double frequency = 1; //频率
    double amplitude = 1; //振幅
    double maxValue = 10.0f;  //用于将结果映射于在[0.0,1.0]的区间内，本文定为10
    for(int i=0;i<octaves;i++) {
        total += perlinNoise(Vector2(x * frequency, y * frequency)) * amplitude;

        maxValue += amplitude;

        amplitude *= persistence;
        frequency *= 2;
```

当然，倍频组数的增加，会线性地增加代码执行时间，在游戏运行时使用噪声算法，再好不要使用超过几组倍频（比如，当你想在60fps下模拟火焰特效时，最好不要这么干）。然而，做数据预处理时，就很适合使用多组倍频叠加来模拟更自然的噪声（比如用于提前生成游戏地形等）。

结语

- 什么时候使用Value噪声和柏林噪声：Value噪声简单性能开销小，但是产生的噪声可能略显机械重复（看起来不自然，容易形成"砖块"，可参考本文第三张图片效果）；而柏林噪声则计算稍加复杂，但是产生的噪声更为自然。也就是说这是在性能和效果之间做取舍。
- 2维的可平铺噪声其实也可以在3维球上采样（因为三维球可以用yaw/pitch两个维度表示三维球面上所有点），而无需每个维度在二倍的维度上采样（导致最终是4维采样）。
- 柏林噪声的随机向量一定需要同样的长度吗：测试中...

实际上本文很多篇幅来源于直接参考的几个文章，这些业界前辈在他们的原文已经写的相当精彩。只是为了简化整理内容和补充细节，我才有了整理出这篇博文的想法。
之后有空再写一篇关于地形生成机制，不过在那之前先好好研究Minecraft和其他类似游戏的地形生成机制，如果有不错的资料（不必是代码实现，只是剖析/概念也可）不妨请告诉我，感激不尽。

参考

- [1] 【图形学】谈谈噪声 - candycat - CSDN博客
- [2] 一篇文章搞懂柏林噪声算法，附代码讲解 - 立航 - 博客园
- [3] 一篇关于simplexnoise和perlinnoise的论文
- [4] Perlin噪声 wiki
- [5] 程序丨游戏编程中的数学：基于噪音的随机数字生成
- [6] 《游戏编程精粹2》 Mark A. Deloura [2003-12-1]

作者：KillerAery 出处：http://www.cnblogs.com/KillerAery/

本文版权归作者和博客园共有，未经作者同意不可擅自转载，否则保留追究法律责任的权利。

分类：游戏算法&数据结构
标签：算法&数据结构

免责声明：本内容来自平台创作者，博客园系信息发布平台，仅提供信息存储空间服务。